

A9 — Camera & Handling

This is the point where the HQ stops being only a well-designed building and becomes something that **feels coherent to play**.

A9 has to solve one central problem:

The camera and controls must adapt to very different activities without making Phantom Q feel like several unrelated games stitched together.

The player will be walking through HQ, talking to Alice, using Workstation 04, arranging evidence, working at a Crypto bench, inspecting a network map, operating inside SOC and conducting authorised Red Team activity.

Those interactions need different viewpoints, but they must all feel like they belong to **one control language**.

A9.1 — Core camera philosophy

I would lock the normal Phantom Q camera as:

```
RAISED OBLIQUE PERSPECTIVE
+
FIXED WORLD ORIENTATION
+
SMOOTH PLAYER FOLLOW
+
CONTEXTUAL CAMERA TRANSITIONS
```

The player should **not continuously control camera rotation** during ordinary HQ movement.

That gives us:

```
W = visually forward/up-screen
S = visually backward/down-screen
A = left
D = right
```

and those directions remain understandable throughout the building.

That consistency matters much more for this game than unrestricted camera freedom.

A9.2 — Why we should not use a normal third-person camera

A conventional over-the-shoulder camera would create several problems:

```
narrow field of view
↓
less awareness of rooms
↓
more camera management
↓
harder navigation
↓
weaker Sims-style architectural view
↓
less useful for teaching systems spatially
```

We deliberately want the learner to see:

- themselves;
- Alice;
- nearby workstations;
- access doors;
- room functions;
- environmental state.

The raised viewpoint gives us that.

A9.3 — Why we should not use pure isometric/orthographic either

Pure orthographic would make the HQ very clean, but it would reduce the visual quality and physical depth we want.

So our normal camera remains **perspective**, with a relatively narrow FOV.

Initial baseline:

```
Pitch          60°
Yaw            45°
Vertical FOV    40°

Initial test range:
```

I would start at exactly:

```
Pitch = 60°  
Yaw   = 45°  
FOV   = 40°
```

and only change it after greybox testing.

A9.4 — Normal exploration camera position

Relative to the player, approximately:

Camera offset

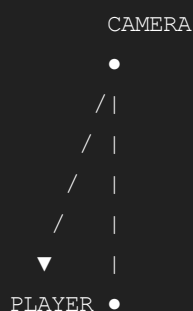
```
X = -8 m  
Y = +11 m  
Z = -8 m
```

giving approximately:

14-16 m

camera-to-player distance.

Conceptually:



This is high enough to understand the environment but still close enough to see character animation.

A9.5 — The camera should not follow the player rigidly

The player may move quickly, but the camera should behave like a stabilised system.

We need:

PLAYER MOVES

↓

camera target moves

↓

camera smoothly catches target

rather than:

PLAYER MOVES

↓

CAMERA IMMEDIATELY TELEPORTS

Initial feel:

Position damping:

~0.15-0.25 s

Rotation:

mostly fixed

Vertical response:

slightly slower than horizontal

Look-ahead:

small

The result should feel **controlled**, not floaty.

A9.6 — Small directional look-ahead

One refinement I would add is subtle look-ahead.

If the player walks north:

PLAYER

↑

CAMERA TARGET

moves slightly ahead

perhaps:

0.5-1.0 m

in the movement direction.

That lets the learner see where they are going without shifting the entire composition dramatically.

When the player stops:

```
camera target  
smoothly returns toward player centre
```

A9.7 — Camera focus should be room-aware

The camera should not blindly follow the player's transform.

Every major room receives a:

Camera Profile

For example:

CENTRAL OPERATIONS

Follow Distance	15 m
Pitch	60°
FOV	40°
Look Ahead	0.8 m
Room Bias	centre
Wall Cutaway	standard

While a smaller room might use:

RECEPTION

Follow Distance	12.5 m
Pitch	62°
FOV	38°
Room Bias	reception centre

And Quantum later:

QUANTUM LAB

Follow Distance	16 m
Pitch	58°
FOV	42°
Room Bias	optical bench

This gives us room-specific polish without changing the player's overall camera language.

A9.8 — We need a camera state system

I would not build camera behaviour as separate random scripts.

Use one state architecture:

```
CAMERA CONTROLLER
|
├─ EXPLORE
├─ ROOM ENTRY
├─ CHARACTER
├─ WORKSTATION
├─ WORKBENCH
├─ BOARD / TABLE
├─ OPERATIONAL CONSOLE
├─ SEQUENCE
└─ RETURN
```

The mission or interaction requests a camera state.

The camera system decides how to transition.

A9.9 — Camera State 1: Explore

This is the default.

PLAYER CONTROL	FULL
MOVEMENT	WASD / stick
CAMERA ROTATION	LOCKED
CAMERA FOLLOW	SMOOTH
INTERACTION	ENABLED
ROOM POPUP	AVAILABLE

Used for most Headquarters navigation.

A9.10 — Camera State 2: Room Entry

We should **not cut to a cinematic every time a player enters a room.**

Instead use a very small framing adjustment.

Example:

```
PLAYER CROSSES  
CRYPTO ROOM BOUNDARY  
↓  
camera target gently biases  
towards room centre  
↓  
CRYPTOGRAPHY LAB  
room identifier appears  
↓  
player keeps moving
```

No loss of control.

No camera cut.

No forced stop.

That keeps the building flowing naturally.

A9.11 — First-time room introduction

On the first ever entry, we can allow a slightly stronger reveal.

For example:

```
CRYPTO DOOR OPENS  
↓  
player crosses threshold  
↓  
camera slightly widens  
↓  
Crypto workbench enters frame  
↓  
room title  
↓  
normal exploration
```

Maybe:

1.0-1.5 seconds

of subtle framing.

But still not a long cinematic.

A9.12 — Camera State 3: Character interaction

Talking to Alice should not switch Phantom Q into a traditional dialogue game where giant heads fill the screen.

We want to preserve:

```
CHARACTER
+
PLAYER
+
ROOM CONTEXT
```

For normal conversations:

```
raised medium 3D framing
```

showing both people.

Something like:

```
ALICE
•
workstation / object
•
PLAYER
CAMERA
```

The exact camera is determined by the interaction marker.

A9.13 — Character interaction flow

Example with Alice at Workstation 04:

```
PLAYER approaches Alice
↓
"Speak to Alice"
```



```
↓
player auto-aligns to marker
↓
Alice turns toward player
↓
camera transitions
↓
conversation
↓
dialogue ends
↓
camera returns
↓
movement restored
```

The player shouldn't continue walking in circles while Alice is speaking.

So movement is temporarily controlled during conversation.

A9.14 — Look-at behaviour

During conversation:

```
Alice head/eyes
→ Player

Player head
→ Alice
```

If Alice references the USB:

```
Alice
↓
brief look towards USB
↓
return towards player
```

This is a small detail with a large quality impact.

A9.15 — Camera State 4: Workstation

Workstation 04 needs its own interaction language because it is such an important persistent object.

When selected:

```
PLAYER
↓
auto-align to WS04_PlayerMarker
↓
character sits / stands correctly
↓
camera moves behind/above shoulder
↓
monitor becomes dominant
↓
workstation UI activates
```

But we should retain enough 3D environment around the monitor that the learner knows:

I am still physically at Workstation 04.

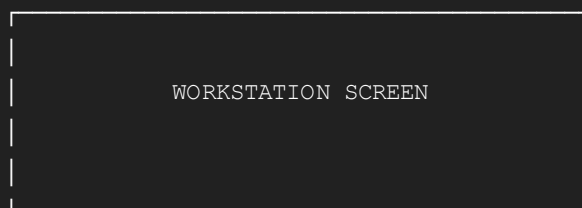
A9.16 — Workstation camera framing

I would avoid:

```
FULL SCREEN
flat fake operating system
```

as the default.

Better:



```
desk edge
keyboard
part of environment
player arm/body edge
```

Then when certain detailed interfaces need more space, the UI can expand.

This maintains world continuity.

A9.17 — Leaving a workstation

Use a consistent Back action.

```
BACK
↓
close application
↓
leave workstation mode
↓
character returns to standing
↓
camera blends to Explore
```

No different escape method for every terminal.

A9.18 — Camera State 5: Workbench

Crypto and Engineering are different.

At a Crypto bench, the learner must manipulate several objects.

So camera should transition to something closer to:

angled tactical tabletop view

Example:

```
      CIPHERTEXT
      •
PLAINTEXT •  MODULE  • KEY A7
      PLAYER
```

Camera:

```
higher pitch
closer distance
workbench centred
```

This makes drag/place/connect interactions easier.

A9.19 — Workbench handling

Player locomotion pauses.

Controls become:

```
Cursor / pointer  
Select  
Drag  
Place  
Inspect  
Back
```

For example:

```
select plaintext  
↓  
place in encryption input  
↓  
select A7  
↓  
place in key slot  
↓  
run
```

We don't require the 3D avatar to physically walk around a 2 m table just to select each item.

That would add friction without adding learning.

A9.20 — Camera State 6: Evidence board / table

Foundations and SOC rely heavily on spatial reasoning.

These need a more top-down view.

For example:

```
Pitch:  
~70°–80°  
  
FOV:  
~35°–40°  
  
Target:  
board/table centre
```

Not fully 90° if we want to retain depth.

The player should be able to:

```
pan slightly
zoom slightly
select
drag
place
compare
```

inside the board.

A9.21 — Evidence board movement

Once engaged:

```
WASD
```

should not accidentally walk the player's body away.

Instead controls switch context.

Desktop example:

Mouse	select/drag
Wheel	controlled zoom
Middle/right	optional pan
Escape/Back	exit

Controller/mobile gets equivalent context controls.

A9.22 — Camera State 7: Communications

Communications is more room-scale.

For many actions we do **not need to lock the camera into a workstation**.

Example:

```
Alice endpoint
  ↓
network route wall
  ↓
Bob endpoint
```

The room itself becomes the interface.

The player can walk between:

```
Source terminal
Transmission console
Bob endpoint
Route display
```

This is important because it makes Communications physically different from Crypto.

A9.23 — Communications route-display interaction

When the learner opens the network route display, camera transitions enough to place the graph centrally.

For example:

```
normal room
↓
Inspect Network Path
↓
camera centres network wall
↓
route nodes become interactive
```

Player body remains positioned at the display.

On exit, back to Explore.

A9.24 — Camera State 8: SOC investigation

SOC needs a mixture.

Normal investigation:

```
EXPLORE
```

to move between stations.

Individual evidence board:

```
BOARD MODE
```

Trace table:

TACTICAL TABLE MODE

Session console:

WORKSTATION MODE

This is one reason our camera architecture must be state-driven rather than room-driven.

SOC contains multiple interaction types.

A9.25 — Network trace table

This deserves a special tactical camera because Incident 06 needs network relationships to be visually understandable.

Example:

PHANTOM Q

```
•
|
|
• 172.18.44.7
|
|
?
```

Camera:

~65-70° pitch
fairly close
network table centred

Then new nodes animate into the table as discovered.

A9.26 — Red Team handling

Red Team should **not suddenly become a first-person shooter**.

That would break everything we have built.

Its operational verbs are:

```
validate  
request  
verify  
inspect  
engage  
search  
preserve
```

Therefore Red Team continues the same interaction grammar.

Normal navigation:

```
raised oblique
```

Target case:

```
board/table camera
```

Authorisation:

```
console camera
```

Controlled access:

```
operational console
```

Artefact search:

```
focused interface
```

Same world.

More advanced decisions.

A9.27 — Controlled attack presentation

When the player selects:

```
INITIATE CONTROLLED ACCESS
```

we should avoid cinematic nonsense like:

green code
screen shaking
red flashing skulls

Instead:

```
AUTHORISATION
VERIFIED

      ↓

TARGET
10.91.24.18

      ↓

SCOPE
REMOTE MANAGEMENT

      ↓

SESSION
ESTABLISHING...

      ↓

CONTROLLED ACCESS
ACTIVE
```

The visual quality comes from:

- precise motion;
- restrained graphics;
- good audio;
- clear system state.

This supports the industrial simulation identity.

A9.28 — Cutaway wall system

This is one of the most important technical camera systems.

Our rule should be:

Never rotate the entire camera simply because one wall blocks the player.

Instead:

```
CAMERA
  \
   \
[WALL]
  \
    PLAYER
```

becomes:

```
upper wall fades/hides  
↓  
player remains visible
```

A9.29 — Wall construction

As discussed earlier, walls should be separable:

```
WALL  
|  
├── LOWER SECTION  
|   0-1.1/1.2 m  
|  
└── UPPER SECTION  
    1.2-3.2 m
```

Then generally:

```
lower wall remains  
upper wall fades/cuts
```

This preserves the architectural outline while exposing gameplay.

A9.30 — Cutaway detection

Conceptually:

```
Camera  
↓  
cast towards player  
↓  
detect CutawayWall objects  
↓  
temporarily fade/hide upper sections
```

When no longer obstructing:

```
smoothly restore
```

Do not instantly pop walls on/off if we can avoid it.

Approximate fade:

0.15-0.30 s

would likely feel good.

A9.31 — Roof/ceiling handling

Because this is a raised camera, ceilings should generally not obscure the room.

We have two options:

A. ceilings not rendered from gameplay camera

or

B. ceiling chunks hide by occupied room

I prefer **room-aware ceiling visibility**.

When outside/establishing shot:

roof may exist

When player enters:

occupied room roof fades away

This gives us both architectural completeness and playable visibility.

A9.32 — Foreground object obstruction

It's not just walls.

Large:

- server racks;
- cabinets;
- screens;
- equipment;

may block the player.

Important foreground objects should support:

camera obstruction fading

rather than forcing the player to disappear behind them.

This is especially important in Engineering and Quantum later.

A9.33 — Player handling baseline

For the first greybox I would use:

Walk speed	3.2 m/s
Fast movement optional / later	4.8-5.0 m/s
Acceleration	10-12 m/s ²
Deceleration	14-16 m/s ²
Character turn	~540°/s
Interaction reach	~1.6-1.8 m

The learner should feel responsive immediately.

Not heavy.

Not twitchy.

A9.34 — Do we need running?

For Release 1, I would **not make sprinting a major mechanic**.

The building is compact.

A fast walk can be sufficient.

If we include a faster movement state:

Walk	3.2 m/s
Fast move	4.8 m/s

it should be convenience, not stamina gameplay.

No stamina meter.

No running challenges.

That would add nothing to Phantom Q.

A9.35 — Character rotation

When moving:

```
character turns toward movement vector
```

but with smoothing.

No instant rotation.

At interaction:

```
character auto-aligns  
toward target
```

For example:

```
player approaches workstation at angle  
↓  
press Interact  
↓  
player takes final 0.5 m  
↓  
turns correctly  
↓  
interaction begins
```

This is a major quality feature.

A9.36 — Contextual auto-path

We should use auto-path **locally**, not as the entire movement system.

Manual movement takes player near an interaction.

Then:

```
INTERACT
↓
if player is inside valid range
↓
auto-path / auto-align
to exact interaction marker
```

Example:

```
player 1.4 m from WS04
↓
Use Workstation
↓
character walks 0.6 m
↓
correctly positions
↓
workstation mode
```

This makes animations line up reliably.

A9.37 — What auto-path should NOT do

Do not make the game:

```
click Crypto
↓
player walks for 12 seconds automatically
```

as the normal control scheme.

That reduces agency.

Contextual pathing should solve:

```
fine positioning
```

not:

```
general navigation
```

A9.38 — Collision quality

Character collision should be forgiving.

We should avoid:

```
chair corner  
↓  
player completely stops
```

or:

```
Alice and player  
lock each other in corridor
```

So furniture colliders should be simplified.

Example:

```
DESK  
simple box collider  
  
CHAIR  
simple low-complexity collider  
  
small decorative objects  
no player collision unless necessary
```

The visible mesh does not need a complex collider.

A9.39 — NPC handling

NPCs use NavMesh.

Typical Alice settings:

Speed	~1.45 m/s
Acceleration	6-8 m/s ²
Angular speed	360-480°/s
Stopping distance	~0.35 m

But we should add:

```
arrival slowdown  
turn-to-target  
look-at  
idle transition
```

so she doesn't look robotic.

A9.40 — NPC/player passing

Main corridors are 3 m partly because we want:

NPC → ← PLAYER

to pass naturally.

NPC navigation should avoid giving the exact centreline to every agent.

A small amount of avoidance produces more organic movement.

A9.41 — Interaction priority

If several interactables are nearby, we need predictable selection.

Priority should consider:

```
active objective
+
distance
+
player facing
+
importance
```

So if Alice and a random chair are beside one another:

Speak to Alice

wins.

The chair probably shouldn't even be interactable.

A9.42 — No interaction clutter

At most one primary prompt should normally appear.

Not:


```
[E] Talk Alice  
[F] Use Desk  
[G] Inspect Monitor  
[Q] Open File  
[R] Sit
```

all at once.

Instead the system determines the most relevant action.

Secondary interactions can be available after focusing/inspection.

A9.43 — Handling room transitions

Doors should be part of movement rather than mini cutscenes.

Player approaches secure door.

If already authorised:

```
reader detects player/interaction  
↓  
brief check  
↓  
door opens  
↓  
player walks through
```

If a significant first unlock:

```
credential check  
↓  
ACCESS GRANTED  
↓  
door opens  
↓  
small audio/visual event
```

No loading screen between adjacent HQ rooms.

A9.44 — Door opening timing

Approximately:

0.8-1.0 s

for normal secure doors.

Red Team/high-security transitions can deliberately feel slightly more substantial:

1.2-1.5 s

because the boundary matters.

But never so slow that repeated use becomes annoying.

A9.45 — Security vestibules

At T2/T4/T5, camera behaviour can help communicate higher security.

Example Red Team vestibule:

```
SOC
↓
first door
↓
player enters vestibule
↓
camera tightens slightly
↓
authorisation state visible
↓
second door opens
↓
camera returns to normal room profile
```

No dramatic cutscene needed.

The space itself creates tension.

A9.46 — Player navigation assistance

We said the HQ should be understandable without a minimap during normal use.

So navigation assistance should be environmental first.

Use:

```
room signage
lighting
door labels
major landmarks
objective direction indicator if needed
```

Only use a full map when helpful, rather than permanently occupying screen space.

A9.47 — Objective direction

If the objective is:

```
GO TO CRYPTOGRAPHY LAB
```

we can show a subtle world-space direction at decision points.

For example:

```
← CRYPTOGRAPHY
```

on existing HQ signage.

Not a giant glowing line across the floor unless accessibility mode enables stronger navigation assistance.

A9.48 — Mobile handling

We should design this now even if PC comes first.

Normal mobile layout:

```
LEFT
virtual movement stick

RIGHT
context interaction button

camera
mostly automatic
```

This is another reason the fixed camera is good.

If we used conventional free-camera third person, mobile would require:

```
left thumb = movement
right thumb = camera
third action = interaction
```

much more demanding.

Our architecture allows:

```
left thumb = movement
right thumb = interact
```

with camera automation doing the rest.

A9.49 — Desktop controls

Keep them simple.

WASD	Move
E / Left-click	
context interaction	
Esc / Right-click	
Back	
Mouse	
UI / workbench manipulation	
Wheel	
controlled zoom where allowed	

We can refine actual mappings later through Unity's Input System.

The important thing is the **small conceptual vocabulary**.

A9.50 — Controller handling

Same abstraction:

Left Stick	Move
Primary Button	Interact
Secondary	Back
D-pad/Stick	contextual UI

We should build against abstract input actions, not hard-code keyboard keys.

A9.51 — Camera zoom

Normal exploration should allow only limited zoom.

Maybe:

Near	~12 m
Default	~15 m
Far	~18 m

This helps accessibility and screen size.

But yaw remains fixed.

The player should not be able to rotate the camera until the HQ becomes visually unreadable.

A9.52 — Camera shake

Very little.

This is not an action shooter.

Avoid gratuitous camera shake for:

- door opening;
- alerts;
- attacks;
- money loss.

Use it only if there is a genuine physical event.

Most incident tension should come through:

sound
environment state
NPC behaviour
display changes

not shaking the screen.

A9.53 — Motion comfort

We need to avoid:

- sudden zooms;
- rapid camera rotation;
- instant FOV changes;
- excessive head bob;
- unnecessary screen shake.

Camera blends should usually be:

0.25–0.6 s

depending on context.

First-time room reveal may take slightly longer.

A9.54 — Interaction camera transitions

I would standardise roughly:

Explore → Character

0.35 s

Explore → Workstation

0.45 s

Explore → Workbench

0.40 s

Explore → Evidence board

0.35 s

Return → Explore

0.30–0.45 s

These are starting points, not permanent magic numbers.

Consistency is more important than exact duration.

A9.55 — Do not make Timeline control gameplay truth

This is important technically.

For scripted sequences, Unity Timeline can control:

```
camera  
animation  
audio  
timing
```

But the mission system should remain responsible for:

```
what actually happened  
what objective is complete  
what state changed
```

So:

```
MISSION LOGIC  
requests  
SEQUENCE  
  
SEQUENCE  
presents event  
  
MISSION LOGIC  
records result
```

not:

```
Timeline happened  
therefore game assumes mission complete
```

This separation will make debugging much easier.

A9.56 — Sequence interruption

Important story sequences should establish whether they are:

```
SKIPPABLE  
or  
NON-SKIPPABLE
```

But even non-skippable sequences should usually remain short.

Prologue Alice walking to WS04:

maybe several seconds

not a two-minute cinematic.

Phantom Q is interaction-led.

A9.57 — Quality through animation blending

Handling quality is not only controller responsiveness.

Character movement needs blending:

```
Idle
↓
Start Walk
↓
Walk
↓
Slow
↓
Stop
↓
Turn
↓
Interact
```

not:

```
Idle → Walk instantly
Walk → Idle instantly
```

Root motion may be useful for specific interaction animations, while general navigation can remain movement-controller/NavMesh driven.

We can decide the exact animation architecture when we build the Character module.

A9.58 — Foot sliding

This will be one of our quality tests.

If the player's speed and animation disagree:

feet slide

and the project immediately feels cheap.

Therefore later:

```
movement speed
↔
Animator locomotion blend
```

must be calibrated together.

Same for Alice.

A9.59 — Interaction camera quality gate

For every important interaction, we check:

```
Can I clearly see the object?

Can I see enough environment
to know where I am?

Is text readable?

Does the player/NPC clip?

Does the transition feel smooth?

Can I exit predictably?
```

If any fails, the interaction is not finished.

A9.60 — Room camera profiles

I would currently map them like this:

Room	Exploration framing	Primary interaction camera
Reception	Slightly closer	Character/identity terminal
Central Ops	Wide/open	Workstation + character
Foundations	Medium	Board/table overhead
Crypto	Medium	Workbench tactical

Room	Exploration framing	Primary interaction camera
Communications	Wide	Network-wall/terminal
SOC	Wide-medium	Evidence/trace tactical
Red Team	Medium	Operational console/table
Engineering	Wide-medium	Bench
Quantum	Wide	Optical bench/lab
Advanced Compute	Medium	Simulation console
Secure Core	Deliberately controlled	Future

This reinforces each room's gameplay identity.

A9.61 — Central Operations camera should be the benchmark

If Central Ops does not feel right, we should **not tune every other room independently**.

Central Ops becomes the base profile:

HQ CAMERA BASELINE

Other rooms modify it slightly.

That keeps the whole project visually coherent.

A9.62 — Handling during incident states

The camera should not radically change because the story becomes serious.

For example Incident 04:

```
normal camera language
+
SOC activity
+
audio
```

```
+  
display state  
+  
lighting subtle change
```

not:

```
camera suddenly becomes shaky  
dark red horror game
```

The Headquarters remains professional.

The situation changes.

A9.63 — Incident 06 should feel more focused without becoming a different game

Red Team may use:

```
slightly tighter room framing  
more focused interaction cameras  
denser operational UI
```

but exploration still obeys the same movement rules.

This lets progression feel more advanced without forcing the player to relearn controls.

A9.64 — Handling and reading load

Because one of our goals is to minimise unnecessary reading, the camera needs to make visual information readable.

That means:

```
object position  
+  
animation  
+  
camera  
+  
state colour  
+  
short verb
```

should often communicate the action before text explains it.

Example:

```
RED TEAM DOOR  
reader amber  
status:  
MISSION AUTHORITY REQUIRED
```

The learner understands the obstacle almost immediately.

A9.65 — Accessibility handling

We should eventually expose:

```
movement sensitivity  
camera follow strength  
zoom level  
text size  
interaction hold/tap  
strong navigation assistance  
reduced motion  
camera transition speed
```

The fixed camera actually makes accessibility easier because we control the visual environment.

A9.66 — Performance implications

The raised camera can see more geometry than conventional first-person views.

So camera design and optimisation are linked.

We need:

```
LOD  
occlusion  
room culling  
cutaway geometry  
controlled sightlines
```

from the start.

This is another reason the hub + short-wings map is useful.

The camera is unlikely to see the entire 108 m building simultaneously.

A9.67 — Dynamic room activation

As the player enters a wing:

CURRENT ROOM

high detail

ADJACENT ROOMS

normal

DISTANT ROOMS

reduced / culled where appropriate

We should not deactivate entire rooms so aggressively that windows suddenly become empty, but we can manage:

- NPC activity;
- small props;
- effects;
- update frequency.

That supports Web/mobile performance.

A9.68 — Quality targets

For handling, I would set:

Desktop

60 FPS target

Web/lower laptop

stable 30-60 FPS

Mobile

stable 30 FPS

But input response must remain immediate even at the lower target.

The game should never feel:

press

↓

wait

↓
character starts moving

Input latency is more damaging to perceived quality than slightly reduced visual settings.

A9.69 — What we should test first in Unity

When we reach the first greybox, the camera/handling prototype should contain only:

```
Central Operations
3 m corridor
one secure door
Player
Alice
Workstation 04
one board/table
```

That gives us enough to test nearly every camera family:

```
Explore
Room entry
Door
Character
Workstation
Board
Return
```

If those work, the camera foundation is credible.

A9.70 — First handling test sequence

The initial Unity prototype should let us perform exactly this:

```
SPAWN IN CORRIDOR
↓
walk towards Central Ops
↓
secure door opens
↓
enter room
↓
room camera adjusts
↓
walk around WS04
↓
Alice enters
```

```
↓
Alice walks to marker
↓
Speak to Alice
↓
conversation camera
↓
return
↓
Use Workstation 04
↓
workstation camera
↓
exit workstation
↓
walk to prototype evidence table
↓
board/table camera
↓
move one object
↓
exit
↓
resume exploration
```

That single prototype would prove almost the entire A9 architecture.

A9.71 — Camera technical hierarchy in Unity

Conceptually:

```
CameraSystem
|
├── CameraStateController
|
├── ExplorationCamera
|
├── RoomCameraProfile
|
├── InteractionCameraController
|
├── CharacterCamera
|
├── WorkstationCamera
|
├── WorkbenchCamera
|
├── BoardCamera
|
```

```
├─ SequenceCamera
|
├─ OcclusionController
|
└─ CutawayController
```

Cinemachine can handle much of the actual blending/framing.

Our C# system decides **which state should currently own the camera**.

A9.72 — Handling technical hierarchy

```
PlayerController
|
├─ Movement
├─ Rotation
├─ InteractionDetector
├─ AutoAlign
├─ ContextInput
└─ ControlState
```

The control state can be:

```
Exploring
Interacting
Workbench
Board
Terminal
Dialogue
Sequence
Locked
```

So the same input cannot accidentally control two things at once.

A9.73 — Camera and controls must share state

This is critical.

When:

```
Camera = Workbench
```

then:


```
Player movement = disabled  
Workbench input = enabled
```

When:

```
Camera = Explore
```

then:

```
Player movement = enabled  
Workbench input = disabled
```

Do not allow the camera and control systems to independently guess what mode the player is in.

They should read the same central interaction state.

A9.74 — Loop against A1–A8

A1 — Building meaning: passes. The camera presents the HQ as one persistent facility rather than disconnected screens.

A2 — Spatial hierarchy: passes. Fixed orientation and progressively deeper wings make the security hierarchy visually understandable.

A3 — Adjacency: passes. Short direct relationships become easy physical transitions instead of loading screens.

A4 — Floor plan: passes. No major relocation required; cutaway-friendly walls and room camera profiles should be built into the greybox.

A5 — Campaign routes: passes. Player movement remains meaningful, while remote commands and messages do not cause unnecessary travel.

A6 — Character routes: passes. NPC pathing and scripted movement work independently from camera framing, with interaction markers providing reliable composition.

A7 — Clearance: passes. Doors, vestibules and access readers can communicate security state without changing the basic movement system.

A8 — Interactions: passes strongly. The state camera system supports the different interaction vocabularies without turning every activity into a flat full-screen interface.

A9 — Canonical lock

I would now lock the fundamental handling language as:

NORMAL WORLD

Raised oblique perspective
Fixed world orientation
Camera-relative movement

MOVEMENT

Manual by default

INTERACTION

Contextual
+
short auto-align / auto-path

ROOM ENTRY

No forced cinematic

CHARACTER

Controlled medium framing

WORKSTATION

Focused 3D monitor framing

WORKBENCH

Angled tactical/table view

EVIDENCE

High-angle board/table view

SOC / RED TEAM

Same core handling,
more specialised interaction states

WALLS

Cut away/fade rather than rotate camera

CAMERA TRANSITIONS

Smooth and restrained

PLAYER

never has to continually fight the camera

And the most important principle:

The controls remain simple while the decisions become progressively more sophisticated.

The learner should not become “better at Phantom Q” because Incident 06 requires ten more buttons. They become better because the same basic interaction language is now being used to **trace, correlate, verify authority, engage, preserve and recover**.

That gives us a consistent game from the first walk through Reception to the final restoration of Central Operations.